Phase 2 – Développement avec IA et synthèse d'équipe

Réalisé par :
Sajith ABDOUL,
Ryan AGIN,
Rania BOUSFIHA,
Lucas FERARD,
Royston GNANAPRAGASAM,
Ciffedinne MAHDJOUB

28/02/2026

Dans le cadre de cette phase 2 pour notre SAE, nous avons utilisé plusieurs intelligences artificielles afin d'assister le développement de l'application de génération de diagrammes de Voronoï.

Les IA utilisées sont :

- ChatGPT (version 5.2)
- Claude
- Deepseek
- Perplexity

1 Les prompts utilisés

Nous avons formulé des prompts précis afin d'obtenir du code structuré et conforme aux contraintes du projet (TDD, architecture propre, SOLID). Ils ont également été progressivement modifiés et ajustés selon les IA pour parvenir à une application fonctionnelle.

Le prompt de base utilisé pour chacune des IA est le suivant :

"Je suis étudiante en 3e année de BUT Informatique et je dois développer une application qui génère un diagramme de Voronoï à partir d'un ensemble de points du plan. L'application doit importer un fichier texte contenant une paire de coordonnées par ligne au format x,y (exemples : 2,4, 5.3,4.5), gérer les espaces et les nombres décimaux, et détecter proprement les erreurs de format. À partir de ces points, elle doit calculer le diagramme de Voronoï, l'afficher dans une interface conviviale et permettre l'export en SVG et en image. L'affichage doit utiliser une couleur différente pour chaque point et montrer, dans l'interface, une liste des points avec un marqueur de couleur correspondant (point coloré, pas un code hex) afin de vérifier facilement chaque coordonnée. Le développement doit respecter des standards professionnels très stricts : TDD, architecture propre et modulaire, principes SOLID, faible couplage, forte cohésion, fonctions courtes, noms entièrement explicites et aucun commentaire inutile. Une suite complète de tests automatisés doit être fournie, avec des tests unitaires et quelques tests d'intégration couvrant l'ensemble du flux de l'application. Réalise moi tout cela en respectant les consignes."

2 Analyse des corrections nécessaires et temps passé

IA CHATGPT

Pour l'IA ChatGPT, nous ne disposons plus de l'historique complet de la conversation. Toutefois, voici le fil conducteur des principaux échanges et ajustements réalisés au cours du développement.

ChatGPT a permis de générer une base de projet complète et structurée. Cependant, plusieurs ajustements ont été nécessaires afin d'obtenir un résultat conforme aux attentes.

Les principales corrections apportées sont les suivantes :

Au départ, un point n'était pas correctement pris en compte dans le calcul du diagramme, ce qui entraînait l'absence d'une arête dans la structure finale. Le diagramme semblait donc incomplet par rapport aux coordonnées fournies. Pour corriger cela, nous avons reformulé le prompt en précisant : « Vérifie que tous les points importés sont utilisés dans le calcul du diagramme de Voronoï et qu'aucun n'est ignoré. Corrige le problème de l'arête manquante. »

Par la suite, des problèmes d'affichage liés à la transformation des coordonnées en pixels sont apparus. Le diagramme était déformé ou mal centré dans la zone d'affichage. Nous avons alors utilisé un prompt plus précis du type : « Corrige la transformation coordonnées en pixels en conservant un ratio 1:1 entre les axes X et Y et centre le diagramme. » Cela a permis d'améliorer la cohérence visuelle, mais d'autres ajustements ont été nécessaires concernant le padding et le rectangle de clipping, qui créaient une zone blanche inutile.

De plus avec le prompt de départ il ne prenait pas en compte tous les cas de test il a fallu lui préciser de les prendre en compte (pour 1 point zéro points ou point négatifs ou chaine de caractère à la place de nombre)

Un problème est aussi apparu lors de l'export en SVG et en PNG : le rendu exporté ne correspondait pas exactement à celui affiché dans l'application. Certaines proportions ou éléments visuels changeaient légèrement. Nous avons donc reformulé la demande en précisant : « Assure-toi que l'export SVG et PNG reproduit exactement le même rendu que celui affiché dans l'interface, sans modification d'échelle ni perte d'éléments graphiques. »

Au total, environ une vingtaine de prompts ont été nécessaires pour obtenir un résultat conforme aux attentes du projet. Le temps consacré aux corrections et aux ajustements est estimé à environ 45 minutes. Cette expérience montre que cette IA permet d'accélérer le développement, mais a besoin de reformulations successives et une validation rigoureuse pour obtenir un résultat précis et fiable.

IA PERPLEXITY

Perplexity a rapidement fourni une structure complète et bien organisée du projet. Mais plusieurs interventions ont été indispensables pour atteindre un résultat pleinement satisfaisant.

Au départ, le générateur de diagrammes présentait des lacunes : avec 2 ou 3 points, il produisait soit une erreur, soit un résultat vide, alors qu'un rendu visuel était attendu (médiatrice pour 2 points par exemple). Plusieurs reformulations précises ont été nécessaires : « Intègre des traitements spécifiques pour 0, 1, 2 et 3 points avec diagrammes raisonnables plutôt que strictement conformes mathématiquement. »

La transformation des coordonnées vers l'affichage pixel a également nécessité de multiples ajustements. Les premières versions généraient un cadrage incorrect (diagramme hors cadre, zoom inadapté, déformation). Il a fallu imposer explicitement : « Maintient un ratio 1:1 entre axes X et Y, centre parfaitement et englobe points et arêtes avec une marge proportionnelle. »

L'incohérence entre affichage en temps réel et exports SVG/PNG était problématique : différentes logiques de calcul de zone engendraient des zooms, centrages ou proportions divergents. La consigne « L'export doit reproduire pixel-par-pixel le rendu de l'interface sans variation d'échelle » a finalement synchronisé les trois rendus, mais a nécessité plusieurs itérations.

Perplexity modifiait souvent sa stratégie entre versions : parfois centrage forcé sur coordonnées hardcodées, parfois padding fixe en pixels, ou encore calculs dynamiques partiels. Cette instabilité technique contredit les principes SOLID de cohérence. Il a fallu imposer une méthode unique et justifiable.

La couverture des tests était insuffisante : les cas limites (coordonnées négatives, très peu de points) et la cohérence affichage/export n'étaient pas validés. Les tests manquants ont dû être ajoutés en lui disant explicitement quoi tester et comment.

Voici le lien pour la conversation avec Perplexity:

Prompts Perplexity

IA CLAUDE

Voici le lien pour la conversation avec claude:

https://claude.ai/share/a4640f73-a586-4002-add1-b36a5972b4aa

Comme consultable via le lien de la conversation avec l'IA fourni ci-dessus, lors de l'envoi du premier prompt, l'IA a fourni une architecture complète et cohérente avec les attentes du projet. Cependant, ayant oublié de mentionner dans le prompt initial le langage souhaité, à savoir Python, les fichiers étaient faits en JavaScript. De ce fait, un deuxième prompt a été envoyé afin de refaire les fichiers proposés dans le bon langage. En recréant à l'identique l'architecture que ClaudeAI a proposé, j'ai pu tester l'application qui, en premier lieu, affichait une erreur : le fichier index.html était introuvable… Ainsi, un problème de chemin a été détecté. J'ai donc voulu corriger cela mais, au moment de demander à l'IA, je me suis rendu compte que la version gratuite était arrivée à échéance pour cette journée. En effet, ClaudeAI avait déjà généré en seulement deux requêtes une grande quantité de fichiers. Je me suis alors rabattu sur une autre IA, à savoir Perplexity, pour un seul prompt où je lui ait donné le fichier qui posait problème, et voilà le résultat obtenu :

corriges moi ce code pour qu il pointe vers /public/index:
def do_GET(self) -> None:
if self.path == "/" or self.path == "/index.html":
self._serve_static_file(PUBLIC_DIR / "index.html", "text/html; charset=utf-8")
else:
self._send_json_error(404, "Route introuvable")

Montrer moins

Voici la correction simple pour `do_GET` :

```python
def do_GET(self) -> None:
    if self.path == "/" or self.path == "/index.html":
        # Chemin ABSOLU vers public/index.html depuis la racine du projet
        index_path = Path.cwd() / "public" / "index.html"
        self._serve_static_file(index_path, "text/html; charset=utf-8")
    else:
        self._send_json_error(404, "Route introuvable")
```

En remplaçant la fonction do_GET initiale par celle fournie dans un second temps, le problème était réglé et l'application fonctionnait correctement, sans nécessité de corriger quoique ce soit d'autre. En effet, en plus d'absences d'autres erreurs, l'architecture proposée respecte bien les bonnes pratiques de programmation, comme détaillée dans le ReadME lié à cette partie, et tous les points d'exigences de l'application sont présents.

De ce fait, la correction de problème m'a pris environ 5 minutes. Si l'on compte le temps total passé pour faire cette application, alors j'estime ce temps à environ 30 minutes, ClaudeAI ayant été assez long pour générer les fichiers (qui étaient assez nombreux) et en comptant aussi le temps de mettre en place l'architecture qu'il proposait et de tester l'application via l'interface et via la batterie de tests.

IA DEEPSEEK

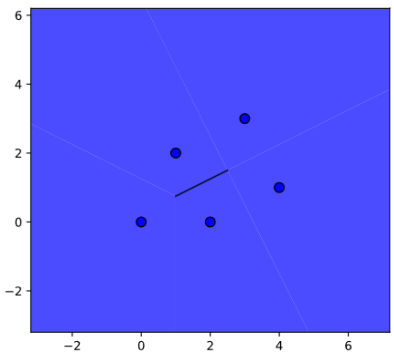
Voici la conversation entre moi et l'IA DeepSeek :
https://chat.deepseek.com/share/z1re9zrtj3dfbxkmtn

Premièrement, l'IA de base que j'avais choisie était Grok, mais suite à ses performances qui n'ont pas pu répondre à ma requête de faire un diagramme de Voronoï, j'ai changé d'IA. C'est pourquoi j'ai choisi la célèbre IA chinoise DeepSeek. J'ai trouvé intéressant de choisir une IA non américaine, contrairement à Claude et à ChatGPT, afin d'avoir un aperçu des avancées technologiques de la Chine durant la nouvelle « Guerre Froide » entre les Etat-Unis et la Chine sur l'avancer technologique des IA et bien d'autre. Après avoir donné le prompt de base, DeepSeek m'a fourni une arborescence claire et précise du projet.

text Copier Télécharger

```text
voronoi_app/
    __init__.py
    model/
        __init__.py
        point.py
        voronoi_diagram.py
    io/
        __init__.py
        point_parser.py
        diagram_exporter.py
    algorithm/
        __init__.py
        voronoi_calculator.py (interface and scipy implementation)
    gui/
        __init__.py
        main_window.py
        point_list_widget.py
        diagram_canvas.py
    app.py (main entry)
tests/
    unit/
        test_point_parser.py
        test_voronoi_calculator.py
        ...
    integration/
        test_full_flow.py
```

avec le contenu des fichiers, mais celui-ci présentait plusieurs problèmes d'import, entre autres. J'ai notamment envoyé une capture d'écran de mon arborescence pour savoir si le problème en était la cause, mais DeepSeek m'a confirmé que non. Je lui ai donc envoyé les erreurs affichées dans mon terminal, et il m'a proposé de créer des fichiers __init__.py dans l'ensemble de mes répertoires afin de résoudre les problèmes d'import et de rendre tous mes répertoires importables en tant que modules. Après la résolution de ces problèmes, l'application se lançait, mais l'affichage ne correspondait pas du tout à ce qu'on attend d'un diagramme de Voronoï. Je lui ai donc demandé de corriger cela. Après une certaine amélioration, le résultat était encore incorrect. J'ai alors essayé d'être le plus précis possible pour que l'IA comprenne mes attentes, en lui envoyant au fur et à mesure des captures d'écran du rendu obtenu avec les défauts constatés. Après avoir finalement obtenu le rendu attendu, j'ai rencontré un problème lors de l'export en SVG et PNG : en effet, les couleurs ne s'affichaient pas correctement, les zones et les points apparaissant de la même couleur.



Il a donc corrigé cela, puis je lui ai demandé de régler quelques détails, comme par exemple que toutes les arêtes séparatrices entre les cellules soient tracées en noir avec une épaisseur visible. J'ai ensuite testé avec plusieurs points et me suis aperçu que l'IA avait ajouté une contrainte exigeant un minimum de 3 points. Je lui ai donc demandé de supprimer cette limitation afin que l'application fonctionne même avec un seul point.

Au total, 26 prompts, également appelés « tours de conversation » par DeepSeek, ont été nécessaires pour obtenir le résultat attendu du diagramme de Voronoï. Le temps que j'y ai consacré est légèrement inférieur à une heure. Cette expérience m'a donné une vision claire de ce que l'IA peut apporter à mon travail : de l'efficacité et de la rapidité, mais avec de nombreux problèmes qu'il faut bien sûr ajuster et corriger au fur et à mesure.

3 Comparaison entre les différentes IA

IA	**Qualité de la première réponse**	**Respect contraintes (smell code)**	**Précision mathématique**	**Rendu app**	**Clarté explication**	**Capacité Correction**	**Temps**	**Nb prompts**	**Note /5**
Chatgpt	Base structurée mais incomplète	Bon	Moyen ajustement nécessaire	Bon après corrections	Très claire	Bonne	45 min	~20	2,5/5

Perplexity	Architecture complete	Bon	Faible - géométrie bancale	Très bien mais classique	claire	Moyenne	50 min	~30	1,5/5
Deepseek	Architecture complète et claire	Bon	Moyen ajustement nécessaire	Très bien mais après de nombreux corrections	claire	Bonne	50 min	26	3,5/5
Claude	Architecture complète, code fonctionnel, pas dans le bon langage	Bon	Aucun ajustement nécessaire	Très bon rendu intiial	ReadmeME très clair et très détaillé	Bonne	30 min	2	4,5/5

4 Comparaison IA vs développement humain

Critère	**IA**	**Développement humain**	**Observations**
Vitesse de production	Très rapide	Largement moins rapide que l'IA mais globalement bonne	L'IA donne une satisfaction de résultat bien plus rapide que l'humain
Gestion des cas limites et tests effectués	Bonne	Relativement bonne	Par exemple, pour les tests des cas peuvent être oubliés par l'humain alors que l'IA pense à tout. Sur ce point, l'IA est largement meilleure c'est indéniable.
Fiabilité mathématique	Correct	Bonne	Pour des algorithmes connus comme celui de Voronoï, l'IA est performante, mais pour des gestion mathématiques plus fines, l'humain a une

			meilleure capacité à raisonner
Qualité architecturale	Très bonne	Bonne	La conception prend plus de temps à l'humain que l'IA. Le résultat est peut-être plus optimisé par l'humain, mais des erreurs sont quand même commises, comme tout mettre dans un seul fichier dans certains cas.
Niveau de bonnes pratiques	Moyenne	Moyenne	L'IA a du mal à mettre en place les bonnes pratiques, ou du moins à les appliquer en même temps. En corriger une va souvent être au détriment d'une autre et on va tourner en rond. L'humain prend, certes, plus de temps, mais le résultat final est souvent meilleur.
Gestion des erreurs	Bonne	Bonne	L'IA les détecte plus rapidement que l'humain mais se trompe parfois sur comment les gérer. Cependant, il en va de même pour les humains et, sur cet aspect là, les deux se valent.
Rendu final de l'app	très bonne	Satisfaisante	L'IA va fournir une application visuellement attrayante en fonction du prompt initial. Cependant, si l'on veut avoir quelque chose de 100% cohérent avec notre volonté, alors l'humain est meilleur. Le temps à

			investir sera plus conséquent, mais le résultat aussi. Reste à prioriser nos importances.

5 Regard critique et analyse globale

Durant cette phase, l'utilisation de plusieurs intelligences artificielles nous a permis d'avancer plus rapidement et d'obtenir assez vite une base fonctionnelle pour notre application de diagramme de Voronoï. Cependant, en travaillant dessus en équipe, nous nous sommes rendu compte qu'il y avait plusieurs limites et risques à ne pas négliger.

Tout d'abord, le code généré peut sembler correct au premier abord, mais certaines erreurs passent facilement inaperçues. Sur des parties un peu complexes comme la gestion des arêtes infinies, les cas de tests ou la transformation des coordonnées en pixels, il pouvait y avoir des imprécisions. Sans vérification de notre part, on aurait pu garder un résultat qui paraissait visuellement correct mais qui était en réalité faux sur le plan mathématique.

Les tests ont aussi montré une limite importante. Même en demandant une approche en TDD (développement dirigé par les tests) avec une suite complète de tests, tous les cas n'étaient pas forcément pris en compte. Par exemple, le diagramme fonctionnait correctement pour plusieurs points, mais dès qu'on testait avec zéro point ou un seul point, le comportement n'était pas toujours géré correctement. Ces cas ne ressortaient pas toujours immédiatement, et il fallait penser à les tester nous-mêmes. Nous avons donc dû reformuler plusieurs fois nos demandes pour insister sur la gestion de tous les cas limites. Cela montre que l'IA peut proposer des tests, mais qu'elle ne pense pas toujours spontanément à tout couvrir.

Nous avons aussi remarqué un décalage entre l'algorithme et l'affichage. Parfois, le calcul du diagramme était correct, mais le rendu visuel ne respectait pas nos consignes. Par exemple, lorsque nous demandions que chaque point soit représenté par un petit marqueur coloré pour bien les différencier, l'IA produisait des points beaucoup trop grands ou mal dimensionnés. Le problème ne venait pas du calcul, mais du rendu graphique. Ce genre de détail n'était pas pris en compte correctement dès le départ et nécessitait plusieurs ajustements.

Un autre point important est la compréhension. Si l'on se contente d'utiliser directement ce que l'IA propose, sans analyser le code, on peut perdre la maîtrise du fonctionnement réel de l'application. Mais dans un projet comme celui-ci, il est essentiel de comprendre les choix techniques et les calculs effectués.

Enfin, nous avons constaté que l'IA ne voit pas le projet dans son ensemble. Elle répond bien à une demande précise, mais elle ne garantit pas toujours la cohérence entre toutes les

parties de l’application, comme le calcul, l’affichage, l’export ou les tests. C’est à nous de vérifier que tout fonctionne correctement ensemble.

En conclusion, l’IA nous a clairement fait gagner du temps et nous a aidé à structurer le projet plus rapidement. Mais elle ne remplace pas notre réflexion, notre vérification et notre compréhension. Pour obtenir un résultat propre et fiable, il a fallu rester attentifs, tester, corriger et reformuler plusieurs fois nos demandes de manière générale.